

5. Strings in JavaScript (Web Developer Level)

Strings are everywhere in web development:

- User names
- Emails
- URLs
- API responses
- Form validation
- Search bars
- Dynamic HTML generation

A strong JS developer should know strings inside out.

1. Creating Strings

```
JavaScriptletstr1="Hello";  
letstr2='Hello';  
letstr3=`Hello`;
```

All are valid.

```
JavaScriptconsole.log(typeofstr1); // string
```

Strings are sequences of characters stored as UTF-16. [\[MDN Web Docs+1\]](#)

2. Accessing Characters

```
JavaScriptletstr="JavaScript";
```

```
console.log(str[0]); // J
console.log(str[4]); // S
```

Using .at()

```
JavaScriptconsole.log(str.at(0)); // J
console.log(str.at(-1)); // t
```

Negative indexing works with at().

3. Length

```
JavaScriptletstr="Hello";
```

```
console.log(str.length);
```

Output:

```
JavaScript5
```

Very commonly used in loops.

```
JavaScriptfor(leti=0;i<str.length;i++){
  console.log(str[i]);
}
```

4. Case Conversion

toUpperCase()

```
JavaScriptletstr="hello";
```

```
console.log(str.toUpperCase());
```

Output:

```
JavaScriptHELLO
```

toLowerCase()

```
JavaScriptletstr="HELLO";
```

```
console.log(str.toLowerCase());
```

Output:

```
JavaScripthello
```

Useful in:

```
JavaScriptemail.toLowerCase()
```

for login systems.

5. Searching Methods

includes()

Checks if substring exists.

```
JavaScriptletstr="JavaScript";
```

```
console.log(str.includes("Script"));
```

Output:

```
JavaScripttrue
```

startsWith()

```
JavaScriptleturl="https://google.com";
```

```
console.log(url.startsWith("https"));
```

Output:

```
JavaScripttrue
```

endsWith()

```
JavaScriptletfile="resume.pdf";  
  
console.log(file.endsWith(".pdf"));
```

Output:

```
JavaScripttrue
```

indexOf()

Returns first occurrence index.

```
JavaScriptletstr="banana";  
  
console.log(str.indexOf("a"));
```

Output:

```
JavaScript1
```

lastIndexOf()

```
JavaScriptletstr="banana";  
  
console.log(str.lastIndexOf("a"));
```

Output:

```
JavaScript5
```

6. Extracting Parts

slice()

Most used method.

```
JavaScriptletstr="JavaScript";
```

```
console.log(str.slice(0,4));
```

Output:

```
JavaScriptJava
```

Negative Indexing

```
JavaScriptconsole.log(str.slice(-6));
```

Output:

```
JavaScriptScript
```

substring()

```
JavaScriptletstr="JavaScript";
```

```
console.log(str.substring(0,4));
```

Output:

```
JavaScriptJava
```

Difference

```
JavaScriptstr.slice(-3);    // "ipt"  
str.substring(-3); // whole string starts from 0
```

Use slice() in modern code.

7. Replacing Text

replace()

Only first occurrence.

```
JavaScriptletstr="hello world";
```

```
console.log(str.replace("world","JS"));
```

Output:

JavaScripthello JS

replaceAll()

```
JavaScriptletstr="a-b-c-d";
```

```
console.log(str.replaceAll("-", " "));
```

Output:

JavaScripta b c d

8. Splitting Strings

Very important.

```
JavaScriptletstr="apple,banana,mango";
```

```
letarr=str.split(",");
```

Output:

```
JavaScript["apple","banana","mango"]
```

Used heavily in:

- CSV data
 - Tags
 - URLs
 - Form inputs
-

9. Joining Strings

```
JavaScriptletarr= ["Java","Script"];
```

```
console.log(arr.join(""));
```

Output:

```
JavaScriptJavaScript
```

10. Trimming Spaces

User inputs often contain unwanted spaces.

```
JavaScriptletemail=" test@gmail.com ";
```

```
console.log(email.trim());
```

Output:

```
JavaScripttest@gmail.com
```

trimStart()

```
JavaScriptstr.trimStart();
```

trimEnd()

```
JavaScriptstr.trimEnd();
```

11. Repeat

```
JavaScriptconsole.log("*".repeat(5));
```

Output:

```
JavaScript*****
```

12. Padding

Useful for formatting.

padStart()

```
JavaScriptletnum="7";
```

```
console.log(num.padStart(3,"0"));
```

Output:

```
JavaScript007
```

padEnd()

```
JavaScriptconsole.log("7".padEnd(3,"0"));
```

Output:

```
JavaScript700
```

13. Concatenation

Old Way

```
JavaScriptletfirst="John";  
letlast="Doe";
```

```
letfull=first+" "+last;
```

Output:

```
JavaScriptJohn Doe
```

14. Template Literals

A must-know feature for modern web development.

Template literals use **backticks (`)** and support interpolation and multiline strings. [\[Devdoc+1\]](#)

```
JavaScriptletname="John";
```

```
console.log(`Hello ${name}`);
```

Output:

```
JavaScriptHello John
```

Multiple Variables

```
JavaScriptletname="John";
```

```
letage=22;
```

```
console.log(`My name is ${name} and I am ${age}`);
```

Output:

```
JavaScriptMy name is John and I am 22
```

Expressions

```
JavaScriptleta=10;
```

```
letb=20;
```

```
console.log(`${a+b}`);
```

Output:

```
JavaScript30
```

Function Calls

```
JavaScriptfunctiongreet(){
```

```
return"Hello";
```

```
}
```

```
console.log(`${greet()} John`);
```

Output:

JavaScriptHello John

Multi-line Strings

```
JavaScriptlettext=`  
line1  
line2  
line3  
`;
```

```
console.log(text);
```

Works directly without `\n`. [\[web.dev+1\]](#)

15. String Immutability

This is the most important concept.

A string **cannot be changed after it is created**. Any operation creates a new string instead. [\[Reddit+1\]](#)

Wrong Thinking

```
JavaScriptletstr="hello";
```

```
str[0] = "H";
```

```
console.log(str);
```

Output:

JavaScripthello

Nothing changes.

Why?

Because strings are immutable.

"hello"

cannot be edited character by character.

Correct Way

```
JavaScriptletstr="hello";
```

```
str="H"+str.slice(1);
```

```
console.log(str);
```

Output:

```
JavaScriptHello
```

A completely new string is created.

Another Example

```
JavaScriptletstr="hello";
```

```
letupper=str.toUpperCase();
```

```
console.log(str);
```

```
console.log(upper);
```

Output:

```
JavaScripthello
```

```
HELLO
```

Original string remains unchanged. [\[Reddit+1\]](#)

16. Common Interview Questions

Reverse String

```
JavaScriptletstr="hello";
```

```
letrev=str.split("").reverse().join("");
```

Check Palindrome

```
JavaScriptletstr="madam";
```

```
letrev=str.split("").reverse().join("");
```

```
console.log(str===rev);
```

Count Characters

```
JavaScriptletstr="banana";
```

```
letcount=0;
```

```
for(letchofstr){  
  if(ch==='a'){  
    count++;  
  }  
}
```

What Every MERN Developer Must Know

Master these 15 methods first:

```
JavaScriptlength
```

```
at
```

```
includes
```

```
startsWith
```

```
endsWith
```

```
indexOf
```

lastIndexOf
slice
substring
replace
replaceAll
split
join
trim
repeat
padStart
padEnd
toUpperCase
toLowerCase

And deeply understand:

1. **Template Literals**
2. **String Immutability**
3. **split() + join()**
4. **slice()**
5. **replace() / replaceAll()**
6. **String ↔ Array conversions**

These six concepts alone appear constantly in React, Node.js, Express, APIs, form validation, and coding interviews. 🚀